

# Plan izrade

## 1. Šta bih uradio

Podelio sam izradu na funkcionalne celine, gde bi cilj bio da se prvo izgradi jedan jednostavniji ali kompletan sistem koji bi pri završetku i krajnjem commit-u na GitHub-u bio označen Tag-om, kao i svako naredno završeno proširenje. Benefit bi bio da se sigurno ostvaruje sve što je strogo traženo u sistemu i bez problema se potom proširuje dodatnim traženim funkcionalnostima. Dodatno, kada dođem do vremenskog limita, lako mogu da se vratim na prošlu verziju i opišem ostatak koji nisam stigao da uradim.

### Plan izrade je sledeći:

- v1.0 - kompletan obavezan sistem (ovde je sve osnovno implementirano ali je svaki bonus uskraćen)
- v1.1 - izolacija privilegija na nivou baze (dodatno pomaže izolaciji podataka)
- v1.2 - robusnost sync-a (dodavanje provera za edge case-ove)
- v1.3 - unapređenje chat sesija (opisano u tekstu zadatka kod extra sekcije)
- v1.4 - dodavanje provajdera storage-a (Google Drive)
- v1.5 - dodavanje provajdera za autentifikaciju korisnika (Clerk)
- v1.6 - kompakcija konteksta (opisano u tekstu zadatka kod extra sekcije)
- v1.7 - unapređenje kvaliteta retrieval-a

Očekujem da ću uraditi obavezan kompletan sistem plus barem još dve dodatne celine. Šta ne stignem da uradim postaje sekcija "sledećih osam sati" u write-up-u.

## 2. Šta bih izbacio i zašto

U v1.0 verziji su izbačeni Clerk i Ory Kratos i zamenjeni sa dva seedovana korisnika za autentifikaciju, iz razloga što je naglašeno da je dovoljno imati dva hardkodovana korisnika za početak (nadogradnja bi bila u tagu v1.5 gde bi se dodao Clerk). Iz provajdera za Datasources su uklonjeni Google Drive i Azure, a ostavljen je samo S3, opet iz razloga zato što je dovoljno za početak (nadogradnja bi bila u tagu v1.4 gde bi se dodao Google Drive). Dodatno, Sync kod ne priča sa S3-om direktno nego preko dogovorenog skupa metoda, pa dodavanje Drive-a znači samo napisati novu klasu koja ih popunjava, bez izmene u ostatku sistema. Pinecone je zamenjen pgvector-om, a LocalStack queues (SQS) Postgres redom poslova, da bi repo radio lakše iz čistog klona i da bi upis da je sync krenuo i predaja tog posla workeru bili jedan isti upis u bazu, pa ne postoji stanje u kom je jedno prošlo a drugo nije. Umesto LangChain orkestracije bih ručno napisao retrieve, prompt, generate petlju, gde je glavni razlog zapravo to što je zadatak jednostavan što se tiče samog RAG sistema i bio bi overengineering da uzmemo LangChain za ovako linearan posao (ali bi definitivno bilo dobro unapređenje za kasniji sistem).

### 3. Tri stvari koje vrede vremena za njihovu izradu

1. Izolacija je fizička, ne filter. Ovo je namerno urađeno iz razloga što sam prednost dao izolaciji. Veća potencijalna greška i rizik bi nam bio da bude narušena privatnost klijenta time što bi neko drugi pristupio njegovim fajlovima, nego cena koju bismo uštedeli za malu šansu da dva korisnika sync-uju potpuno ista dva fajla.

2. Deduplikacija je po korisniku, namerno. "Isti fajl" znači SHA-256 sirovih bajtova, ne putanja, ime ili vreme izmene. Ponovni sync nepromenjenog direktorijuma košta jedan LIST poziv: bez preuzimanja, bez ekstrakcije, bez embedovanja, bez upisa. Između korisnika nikad ne delim embedding, čak ni za bajt-identične fajlove.

3. Sync je state machine, ne dugme. Sync traje, pa se u međuvremenu može desiti drugi klik, osvežavanje stranice ili gašenje procesa na pola. Stanje posla živi u bazi, a sama baza ne dozvoljava dva aktivna posla za isti direktorijum, pa drugi klik dobija "već je u toku" bez ijedne provere u kodu. Napredak se upisuje posle svakog fajla, tako da ugašen proces nastavlja odakle je stao, i taj nastavak skoro ništa ne košta jer se već obrađeni fajlovi preskaču.

### 4. Nejasnoće (Pitanja)

- Mogu li se dva korisnika povezati na isti bucket sa istim kredencijalima?
- Kod ponovnog sync-a izmenjenog fajla, da li zameniti ili čuvati verzije?
- Da li OpenRouter koristim samo za LLM model ili i za pretvaranje teksta u embeddings?

Od kredencijala bi mi trebao OpenRouter i opciono Clerk.